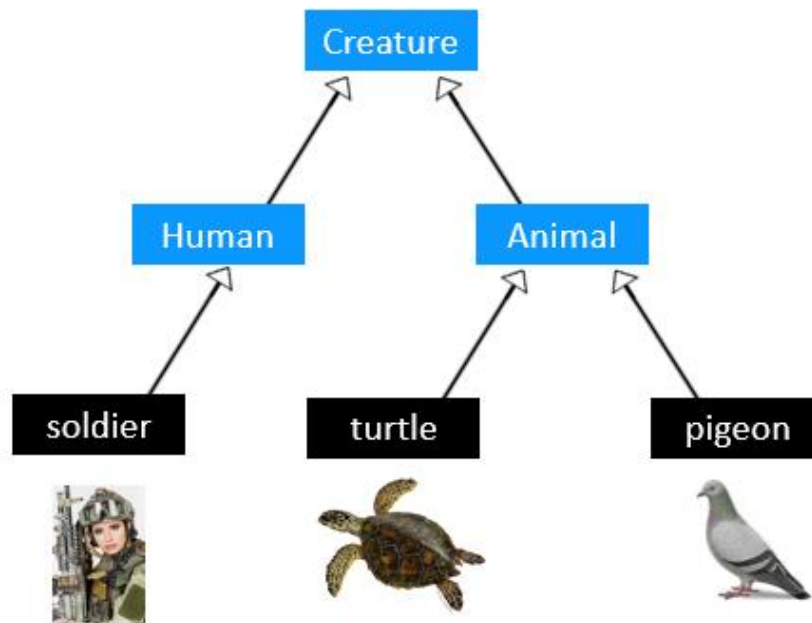


Java2: Lesson 4 – Lab Project

1. Add **summary** at the top, **documentation** at the bottom and **comments** where necessary.
- Recordings and interviews are an exception, but they should be included in your submission code.
2. **Cite** the source of your information.
3. Refrain from including anything **you don't fully understand**.

2	100 points Create a Car and a Bike class by utilizing the interface to generate the following sample output: - Submit a single file containing the test code, StartnStop interface, Car class and Bike class. - Apply your preferred coding style, including indentation and comments, for your final submission.
	<pre>interface StartnStop { void start(); void stop(); } public class Main { public static void main(String[] args) { ... car.start(); car.stop(); bike.start(); bike.stop(); } }</pre>
Sample output	Car started... Car stopped. Bike started... Bike stopped.
File(s) to submit	J204_2.java J204_2.png
3	50 points Modify #1 by adding a CanFly interface to generate the following sample output: Submit a single file containing the test code, CanFly interface, Dog class and Cat class
	<pre>interface CanFly { void fly(); }</pre>
Sample output	Chirp chirp! Bird is flying Woof! Meow!
File(s) to submit	J204_3.java J204_3.png



name	G.I.Jane
age	31
rank	sergeant
job	soldier
species	Human being
attack()	"like a bee"
eat()	"everything..."
shoot()	"shooting..."
swim()	"over 5 km"
talk()	"talking..."

name	Nimo
age	70
habitat	sea
species	Turtle
attack()	"biting..."
eat()	"shrimp..."
swim()	"over 100,000 km"
sound()	"ninja ninja"
hide()	"hiding..."

name	Kiwi
age	2
habitat	land
species	Pigeon
attack()	"pecking..."
eat()	"worm..."
fly()	"flying..."
sound()	"goo goo"
nest()	"nesting..."

Refer to the diagram and instantiate objects for soldier, turtle and pigeon to produce an output.

- Create **abstract classes** (in blue) and **concrete** classes (in black).
- Utilize **interfaces**.
- Declare each variable and method in an appropriate place, accompanied by **justifications** in the **documentation** at the bottom of your code.
- Create a simplified UML diagram for each class and interface in the documentation, using underlines instead of boxes: (<https://www.uml-diagrams.org/class-reference.html>)
- Write the code in a single file, including the test code to generate the similar sample output.
- Structure your code in a way that allows for the seamless integration of multiple concrete classes in the future.

Sample output

```

G.I.Jane: talking...
G.I.Jane: eating...
G.I.Jane: swim over 5 km
Nimo: eating shrimp...
Nimo: swim over 100,000 km
Kiwi: goo goo
Kiwi: pecking...
.....
  
```

	Sample documentation	soldier (concrete → Human) <pre> ----- _*** _***** ----- + ****() + **() ----- Attributes: *** pertain exclusively to soldiers Methods: not all humans might ****() so only **(), ****(). Interfaces: Both *** and *** have the interface *** allowing them have this ability without requiring inheritance. Adding additional classes like *** can easily implement this interface.</pre>
	File(s) to submit	J204_4.java J204_4.png
5	50 points Calculate the multiplication and print out the result in integer. $3 \times 6 \times 9 \times \dots \times 99 = ?$	
	Sample output	3 x 6 x 9 x ... x 99 = 123456789
	File(s) to submit	J204_5.java J204_5.png